



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

Instytut Informatyki

Praca dyplomowa magisterska

ODKRYWANIE ZALEŻNOŚCI MIĘDZY OBIEKTAMI W BAZIE DANYCH

Maria Nowicka, 151851

Promotor

prof. dr hab. inż. Robert Wrembel

POZNAŃ 2026

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP,
w pozostałych kopiach wstawiamy ksero.

Streszczenie

Data lineage i data provenance to pojęcia opisujące śledzenie przepływu danych i historię transformacji danych w systemach bazodanowych. Grafy lineage reprezentują obiekty bazodanowe — tabele, widoki, kolumny — jako węzły, a transformacje danych jako krawędzie. Znajomość zależności między obiektami w bazie danych jest kluczowa z punktu widzenia zapewnienia jakości danych, audytu procesów ekstrakcji, transformacji i ładowania danych (ang. *Extract, Transform, Load*, ETL), debugowania potoków danych oraz spełnienia wymogów regulacyjnych takich jak RODO.

Głównym problemem rozważanym w niniejszej pracy jest zjawisko tzw. *broken lineage* — zerwania ciągłości grafu zależności. Do zerwania ciągłości dochodzi, gdy w procesach przetwarzania danych używane są tabele tymczasowe lub funkcje definiowane przez użytkownika (ang. *User-Defined Function*, UDF). Tabele tymczasowe nie są rejestrowane w standardowych słownikach bazy danych, a UDF ukrywają wewnętrzną logikę transformacji. W obu przypadkach automatyczne odtworzenie pełnego grafu zależności nie jest możliwe — w grafie lineage pojawiają się brakujące krawędzie, których nie można ustalić na podstawie dostępnych metadanych.

Istniejące narzędzia klasy data governance umożliwiają śledzenie lineage w czasie rzeczywistym, lecz wymagają dostępu do pełnych metadanych i nie rozwiązują problemu brakujących krawędzi. W literaturze naukowej podejmowane są próby rozwiązania tego problemu przy użyciu analizy logów systemowych, przepisywania zapytań SQL oraz uczenia maszynowego. Dotychczasowe podejścia nie obejmują jednak analizy wpływu struktury grafu lineage na dobór algorytmów predykcji. Grafy lineage mają charakter dwudzielny — krawędzie przepływu danych łączą wyłącznie węzły tabela i zadanie ETL — co powoduje, że klasyczne heurystyki grafowe oparte na wspólnych sąsiadach są w tym kontekście nieskuteczne.

W ramach niniejszej pracy zaimplementowano i porównano trzy grupy algorytmów predykcji brakujących krawędzi w grafach lineage: heurystyki grafowe (Common Neighbors, Jaccard, Adamic-Adar, Preferential Attachment, L3), algorytmy uczenia maszynowego oparte na cechach topologicznych grafu (Random Forest, RUSBoost, LightGBM) oraz metody zanurzeń sieciowych (*network embeddings*) reprezentowane przez Node2Vec. Tę ostatnią grupę celowo odróżniono od grafowych sieci neuronowych (GNN), z którymi bywa mylona — Node2Vec uczy reprezentacji wektorowych węzłów poprzez biased random walks, bez mechanizmu propagacji wiadomości (*message passing*) charakterystycznego dla GNN. Algorytmy poddano ocenie z użyciem metryk F1, precyzji, czułości oraz AUC-ROC i AUC-PR na zbiorze danych DLG-DG-23, zawierającym 18 grafów lineage ze środowiska Huawei Cloud.

Słowa kluczowe: data lineage, data provenance, broken lineage, predykcja krawędzi, uczenie maszynowe, grafy dwudzielne, Node2Vec, RUSBoost.

Abstract

Data lineage and data provenance refer to tracking data flow and transformation history in database systems. Lineage graphs represent database objects — tables, views, columns — as nodes and data transformations as edges. Knowledge of dependencies between database objects is essential for data quality assurance, auditing Extract-Transform-Load (ETL) processes, debugging data pipelines, and compliance with regulations such as GDPR.

The main problem addressed in this thesis is the phenomenon of *broken lineage* — gaps in the continuity of the dependency graph. These gaps occur when data processing pipelines use temporary tables or user-defined functions (UDFs). Temporary tables are not registered in standard database catalogs, and UDFs hide the internal transformation logic. In both cases, automatic reconstruction of the full dependency graph is not possible — the lineage graph contains missing edges that cannot be determined from available metadata.

Existing data governance tools support real-time lineage tracking, but they require access to complete metadata and do not address the problem of missing edges. Research literature attempts to solve this problem using system log analysis, SQL query rewriting, and machine learning. However, existing approaches do not analyze the impact of lineage graph structure on algorithm selection. Lineage graphs have a bipartite structure — data-flow edges connect only table-job node pairs — which makes classical neighborhood-based graph heuristics ineffective in this context.

In this thesis, three groups of algorithms for predicting missing edges in lineage graphs are implemented and compared: graph heuristics (Common Neighbors, Jaccard, Adamic-Adar, Preferential Attachment, L3), machine learning algorithms based on topological graph features (Random Forest, RUSBoost, LightGBM), and a network embedding method represented by Node2Vec. The latter group is deliberately distinguished from Graph Neural Networks (GNNs) with which it is sometimes confused — Node2Vec learns node representations via biased random walks, without the message-passing mechanism characteristic of GNNs. The algorithms are evaluated using F1, precision, recall, AUC-ROC, and AUC-PR metrics on the DLG-DG-23 dataset, comprising 18 lineage graphs from a Huawei Cloud environment.

Keywords: data lineage, data provenance, broken lineage, link prediction, machine learning, bi-

partite graphs, Node2Vec, RUSBoost.

Spis treści

Streszczenie	iii
Abstract	I
1 Wstęp	1
1.1 Cel i zakres pracy	1
1.2 Struktura pracy	2
2 Podstawy teoretyczne i przegląd literatury	3
2.1 Data lineage i data provenance	3
2.2 Struktura grafu lineage w zbiorze DLG-DG-23	3
2.3 Niekompletność grafów lineage — broken lineage	4
2.4 Predykcja brakujących krawędzi	4
2.5 Metryki ewaluacji	5
2.6 Przegląd literatury	5
3 Problem i sposób jego rozwiązania	7
3.1 Charakterystyka danych	7
3.2 Scenariusze symulacji broken lineage	7
3.3 Zaimplementowane algorytmy	7
3.3.1 Heurystyki grafowe	7
3.3.2 Klasyczne metody uczenia maszynowego	7
3.3.3 Zanurzenia sieciowe — Node2Vec	7
3.4 Architektura rozwiązania	7
4 Ocena rozwiązania	8
4.1 Wyniki eksperymentów	8
4.2 Analiza wyników	8
5 Uwagi końcowe	9
5.1 Podsumowanie	9
5.2 Kierunki dalszych badań	9

Rozdział 1

Wstęp

Współczesne systemy bazodanowe przetwarzają dane za pośrednictwem złożonych potoków transformacji SQL. W środowiskach korporacyjnych dane są wielokrotnie agregowane i przekształcane przez sekwencje zapytań, a wyniki pośrednie przechowywane w tabelach tymczasowych. Tymczasowość tych obiektów sprawia, że po zakończeniu procesu przetwarzania ślad po wykonanych transformacjach zanika — niemożliwe staje się ustalenie, skąd pochodzi dana wartość w tabeli wynikowej ani jakie operacje ją wytworzyły. Zjawisko to, określane w literaturze jako *data lineage* bądź *data provenance*, stanowi istotny problem zarówno z perspektywy audytu danych, jak i ich jakości [?, ?].

Formalnie problem ten można wyrazić w kategoriach teorii grafów: graf zależności (ang. *lineage graph*) reprezentuje przepływ danych między obiektami bazy — tabelami, widokami, zapytaniami i ich atrybutami. Gdy pewne węzły lub krawędzie zostaną utracone (np. z powodu usunięcia tabeli tymczasowej lub zastosowania funkcji definiowanej przez użytkownika), mówi się o zjawisku *broken lineage* [?]. Celem niniejszej pracy jest zastosowanie algorytmów predykcji brakujących krawędzi w celu odtworzenia przerwanych połączeń w grafie lineage.

W sektorze finansowym banki są zobowiązane do wykazania, które transakcje źródłowe złożyły się na poszczególne pozycje raportów kwartalnych; przerwanie grafu zależności w warstwie ETL uniemożliwia przeprowadzenie takiego audytu. Podobne konsekwencje niesie Rozporządzenie o Ochronie Danych Osobowych (RODO): brak pełnego grafu lineage sprawia, że dotarcie do wszystkich kopii danych konkretnej osoby jest czasochłonne lub niemożliwe, co może skutkować nałożeniem kar finansowych.

1.1 Cel i zakres pracy

Celem pracy jest zbadanie skuteczności istniejących algorytmów predykcji brakujących krawędzi w grafach lineage. Zakres obejmuje trzy klasy metod:

- heurystyki grafowe (ang. *graph heuristics*),
- klasyczne metody uczenia maszynowego (ang. *Machine Learning*, ML),
- embeddingi węzłów grafu (ang. *node embeddings*).

Eksperymenty przeprowadzono na zbiorze danych DLG-DG-23, obejmującym 18 zanonimizowanych grafów dwudzielnych ze środowiska Huawei Cloud [?]. Przyjęto trzy scenariusze symulacji brakujących krawędzi, odzwierciedlające typowe przypadki broken lineage:

- **Scenariusz A** — losowe usunięcie 20% krawędzi ze zbioru treningowego,

- **Scenariusz B** — symulacja ukrycia zależności przez tabelę pośrednią (staging),
- **Scenariusz C** — symulacja ukrycia krawędzi do tabeli docelowej (data mart).

Zakres pracy nie obejmuje opracowania autorskiego algorytmu predykcji, odtworzenia rzeczywistego schematu bazy danych ani analizy treści zanonimizowanych danych.

1.2 Struktura pracy

Praca składa się z pięciu rozdziałów. Rozdział 2 zawiera przegląd pojęć podstawowych — data lineage, data provenance, broken lineage oraz predykcja krawędzi — a także omówienie wybranych prac z literatury i stosowanych metryk ewaluacji. Rozdział 3 opisuje problem badawczy, architekturę zaproponowanego rozwiązania, metodykę symulacji brakujących krawędzi oraz zaimplementowane algorytmy. Rozdział 4 prezentuje wyniki eksperymentów przeprowadzonych na zbiorze DLG-DG-23 wraz z ich analizą. Rozdział 5 zawiera wnioski końcowe, podsumowanie uzyskanych wyników oraz propozycje dalszych kierunków badań.

Rozdział 2

Podstawy teoretyczne i przegląd literatury

Niniejszy rozdział wprowadza pojęcia podstawowe niezbędne do zrozumienia dalszych części pracy. W sekcji 2.1 zdefiniowano pojęcia *data lineage* i *data provenance*. Sekcja 2.2 opisuje strukturę grafów lineage na przykładzie zbioru danych DLG-DG-23. Sekcja 2.3 omawia przyczyny niekompletności grafów, a sekcja 2.4 przedstawia problem predykcji krawędzi. Sekcja 2.5 opisuje metryki ewaluacji, a sekcja 2.6 zawiera przegląd wybranej literatury.

2.1 Data lineage i data provenance

Pojęcia *data lineage* i *data provenance* są w literaturze stosowane zarówno zamiennie, jak i z różnieniem semantycznym [?]. W węższym ujęciu *data lineage* odnosi się do śledzenia przepływu informacji przez kolejne etapy przetwarzania — opisuje, przez jakie transformacje przeszły dane i jak zmieniała się ich postać. Z kolei *data provenance* kładzie nacisk na pochodzenie i autentyczność danych: rejestruje, skąd dane zostały pobrane i kto je wytworzył. W praktyce granica między tymi pojęciami jest płynna i większość opracowań traktuje je jako synonimy [?]. Takie podejście przyjęto również w niniejszej pracy.

Znajomość historii danych ma istotne znaczenie praktyczne. W systemach raportowych możliwość odtworzenia ścieżki transformacji pozwala zlokalizować źródło błędnych wartości — bez tej informacji ręczna weryfikacja wielkich zbiorów danych jest nieefektywna. Innym przykładem jest bezpieczne usuwanie obiektów bazy: tabela pozornie niepowiązana z żadnym procesem może w rzeczywistości dostarczać danych do raportów końcowych, a jej usunięcie spowoduje utratę danych bez widocznego komunikatu błędu [?].

2.2 Struktura grafu lineage w zbiorze DLG-DG-23

Zbiór danych DLG-DG-23, opracowany przez Chen et al. [?], zawiera 18 grafów lineage pobranych ze środowiska produkcyjnego Huawei Cloud. Dane zostały zanonimizowane — nie udostępniono schematów tabel. Grafy różnią się skalą: liczba węzłów waha się od 278 do 17 085.

Każdy graf zawiera trzy typy węzłów:

- **job** — zapytanie SQL lub zadanie ETL (ang. *Extract, Transform, Load*),
- **table** — tabela lub widok przechowujący dane,
- **field** — atrybut (kolumna) należący do tabeli.

Krawędzie grafu przyjmują dwa typy:

- **data_flow** — krawędź skierowana między tabelą a jobem lub w kierunku odwrotnym; reprezentuje przepływ danych,
- **parent_child** — krawędź między tabelą a jej atrybutem (polem); wyraża relację przynależności kolumny do tabeli.

Graf jest dwudzielny w tej części, w której węzły job i table tworzą rozłączne zbiory połączone wyłącznie krawędziami data_flow. Właściwość dwudzielności odzwierciedla strukturę przetwarzania ETL: dane przepływają między magazynami danych (table) a procesami transformacji (job).

2.3 Niekompletność grafów lineage — broken lineage

Graf lineage bywa niekompletny z kilku powodów. Tabele tymczasowe, tworzone na potrzeby pojedynczej sesji lub procesu ETL, znikają po jego zakończeniu. Jeśli pośredniczyły w przepływie danych między dwoma trwałymi tabelami, znika wraz z nimi odpowiedni fragment grafu — krawędź reprezentująca tę zależność nigdy nie zostaje utrwalona w metadanych. Analogiczny skutek mają widoki zmaterializowane tworzone tymczasowo oraz funkcje definiowane przez użytkownika (ang. *User-Defined Function*, UDF), które opakowują logikę transformacji bez jawnego zapisu przepływu danych [?].

Zjawisko to, określane jako *broken lineage*, prowadzi do sytuacji, w której graf zawiera węzły bez połączeń lub z niepełnym sąsiedztwem, mimo że rzeczywista zależność między odpowiadającymi im obiektami istnieje. W niniejszej pracy problem broken lineage jest modelowany jako zadanie predykcji brakujących krawędzi — każdej usuniętej krawędzi odpowiada para węzłów, dla której algorytm ma ocenić prawdopodobieństwo istnienia połączenia [?].

2.4 Predykcja brakujących krawędzi

Predykcja brakujących krawędzi (ang. *link prediction*) to zadanie uczenia maszynowego polegające na ocenie prawdopodobieństwa istnienia krawędzi między parą węzłów w grafie [?]. W klasycznym ujęciu problem ten pojawia się w grafach, w których część krawędzi nigdy nie została zaobserwowana. W niniejszej pracy przyjęto odmienne podejście: dysponując pełnymi grafami lineage, krawędzie są usuwane w sposób kontrolowany zgodnie z przyjętymi scenariuszami symulacji (rozdział 3), a zadaniem algorytmów jest ich odtworzenie.

W pracy zastosowano trzy klasy algorytmów:

1. **Heurystyki grafowe** — miary podobieństwa sąsiedztwa węzłów, takie jak Common Neighbors, indeks Jaccarda i miara Adamic-Adar; nie wymagają uczenia, opierają się wyłącznie na strukturze grafu.
2. **Klasyczne metody ML** — Random Forest, RUSBoost i LightGBM; wykorzystują wektory cech opisujących pary węzłów, w tym cechy strukturalne grafu i metryki podobieństwa nazw (np. Jaro-Winkler).
3. **Zanurzenia sieciowe** (ang. *network embeddings*) — metoda Node2Vec, która odwzorowuje węzły grafu w przestrzeni wektorowej tak, by węzły o podobnym kontekście strukturalnym były reprezentowane przez zbliżone wektory; predykcja krawędzi odbywa się przez klasyfikator (MLP) operujący na iloczynie Hadamarda embeddingów pary węzłów. Należy podkreślić, że Node2Vec nie jest grafową siecią neuronową (GNN) — uczy reprezentacji wyłącznie

poprzez biased random walks, bez mechanizmu propagacji wiadomości (*message passing*). Rozszerzenie pracy o właściwą rodzinę GNN (np. R-GCN, HGT) wskazano w rozdziale podsumowującym jako kierunek dalszych badań.

2.5 Metryki ewaluacji

Do oceny jakości predykcji krawędzi stosuje się standardowe metryki klasyfikacji binarnej. W niniejszej pracy przyjęto następujące metryki:

- **Precision** (precyzja) — frakcja poprawnie przewidzianych krawędzi spośród wszystkich krawędzi wskazanych przez model,
- **Recall** (czułość) — frakcja poprawnie wykrytych krawędzi spośród wszystkich rzeczywiście brakujących krawędzi,
- **F1** — średnia harmoniczna precyzji i czułości; główna miara porównawcza stosowana w pracy,
- **AUC-ROC** — pole pod krzywą ROC; mierzy zdolność modelu do rozróżnienia par połączonych od niepołączonych.

Metryka F1 jest szczególnie istotna w warunkach niezrównoważonych klas, charakterystycznych dla grafów rzadkich, gdzie liczba potencjalnych brakujących krawędzi wielokrotnie przewyższa liczbę krawędzi rzeczywistych [?, ?].

2.6 Przegląd literatury

Niniejsza sekcja omawia siedem prac wybranych w ramach systematycznego przeglądu literatury (ang. *Systematic Literature Review*, SLR).

Boiński et al. [?] zaproponowali zastosowanie klasycznych metod ML do problemu broken lineage. Autorzy zastosowali algorytm RUSBoost (ang. *Random Under-Sampling Boosting*) na cechach obejmujących metadane tabel oraz miary podobieństwa nazw (Jaro-Winkler), uzyskując wartość F1 równą 0,79. Praca ta stanowi bezpośrednie odniesienie metodologiczne dla niniejszej pracy.

Chen et al. [?] udostępnili zbiór danych DLG-DG-23 — 18 grafów lineage pobranych ze środowiska produkcyjnego Huawei Cloud, obejmujących grafy o liczbie węzłów od 278 do 17 085. Zbiór ten stanowi podstawowy materiał eksperymentalny niniejszej pracy.

Dietrich et al. [?] omówili specyfikę śledzenia danych w SQL z rekurencyjnymi wspólnymi wyrażeniami tabelowymi (ang. *Common Table Expression*, CTE). Autorzy zaproponowali technikę przepisywania zapytań (ang. *query rewriting*) umożliwiającą propagację informacji o proveniencji przez operacje rekurencyjne.

Kohan Marzagão et al. [?] zaproponowali jądro grafowe (ang. *Provenance Graph Kernel*, PGK) do porównywania grafów proveniencji. Podejście to umożliwia pomiar podobieństwa między grafami lineage bez konieczności ich jawnego wyrównywania.

Psallidas et al. [?] opracowali system OneProvenance, który ekstrahuje informacje o proveniencji danych bezpośrednio z logów silnika bazy danych, budując strukturę drzewa zapytań (ang. *QQTree*) bez modyfikacji kodu źródłowego systemu.

Shan et al. [?] zaproponowali model KPI-HGNN oparty na heterogenicznej grafowej sieci neuronowej (ang. *Heterogeneous Graph Neural Network*, HGNN). Model stosuje mechanizm wykrywania społeczności (ang. *community detection*) w celu agregacji informacji o lokalnym sąsiedztwie węzłów.

Yamada et al. [?] rozszerzyli standardowe podejścia do data lineage o obsługę funkcji definiowanych przez użytkownika (UDF). Autorzy zaproponowali mechanizm rozszerzonych adnotacji lineage (ang. *augmented lineage*), który pozwala śledzić przepływ danych przez ciała funkcji UDF niewidocznych w standardowych planach zapytań.

Rozdział 3

Problem i sposób jego rozwiązania

Niniejszy rozdział opisuje architekturę zaproponowanego rozwiązania, pochodzenie i charakterystykę danych, metodykę symulacji zjawiska broken lineage oraz szczegóły implementacji zrealizowanych algorytmów.

3.1 Charakterystyka danych

3.2 Scenariusze symulacji broken lineage

3.3 Zaimplementowane algorytmy

3.3.1 Heurystyki grafowe

3.3.2 Klasyczne metody uczenia maszynowego

3.3.3 Zanurzenia sieciowe — Node2Vec

3.4 Architektura rozwiązania

Rozdział 4

Ocena rozwiązania

Niniejszy rozdział przedstawia wyniki eksperymentów przeprowadzonych zgodnie z metodologią opisaną w rozdziale 3. Dla każdego z trzech scenariuszy symulacji oraz każdego z trzech algorytmów podano wartości metryk F1, Precision, Recall i AUC-ROC uśrednione po wszystkich 18 grafach zbioru DLG-DG-23.

4.1 Wyniki eksperymentów

4.2 Analiza wyników

Rozdział 5

Uwagi końcowe

Niniejszy rozdział zawiera podsumowanie uzyskanych wyników, wnioski dotyczące skuteczności zastosowanych algorytmów oraz propozycje dalszych kierunków badań.

5.1 Podsumowanie

5.2 Kierunki dalszych badań

Literatura

- [1] Tymoteusz Boiński et al. Machine Learning for Broken Lineage Detection in Relational Databases. In *Proceedings of the ISD 2025 Conference*, 2025.
- [2] Yun Chen et al. DLG-DG-23: A Large-Scale Data Lineage Graph Dataset from Huawei Cloud. 2023.
- [3] Stefan Dietrich et al. Data Provenance for Recursive SQL with Common Table Expressions. 2023.
- [4] David Kohan Marzagão et al. Provenance Graph Kernel for Comparing Data Provenance Graphs. 2023.
- [5] Fotis Psallidas et al. Provenance in Databases: Past, Present, and Future. In *IEEE Data Engineering Bulletin*, 2020.
- [6] Caihua Shan et al. KPI-HGNN: KPI Detection with Heterogeneous Graph Neural Network. 2022.
- [7] Yuu Yamada et al. Augmented Data Lineage for User-Defined Functions. In *Proceedings of the VLDB Endowment*, 2023.



© 2026 Maria Nowicka

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.